

Name : Vivek Singh
RegNumber : 230905408
Roll Number: A50

Lab 4: CONSTRUCTION OF SYMBOL TABLE

1. Using getNextToken() implemented in Lab No 3, design a Lexical Analyser to implement the following symbol tables.
 - a. local symbol table
 - b. global symbol table

Helper functions:(lib.c)

```
/* lib.c */ #include <stdio.h> #include <ctype.h> #include <string.h>

/* ===== TOKEN STRUCT ===== */

typedef struct { char lexeme[50]; char type[30]; int row; int col; } Token;

/* ===== KEYWORD CHECK ===== */

int isKeyword(char *w) { char *k[] = { "auto", "break", "case", "char", "const", "continue", "default",
"do", "double", "else", "enum", "extern", "float", "for", "goto",
"if", "int", "long", "register", "return", "short", "signed",
"sizeof", "static", "struct", "switch", "typedef", "union", "unsigned", "void", "volatile", "while" };

int l = 0, r = 31, m, c;

while (l <= r) {
    m = l + (r - l) / 2;
    c = strcmp(w, k[m]);
    if (c == 0) return 1;
    if (c < 0) r = m - 1;
    else l = m + 1;
}
return 0;

}

/* ===== COMMENT REMOVAL ===== */

void removeComments(FILE *src, FILE *dst) { char c, next;

while ((c = fgetc(src)) != EOF) {

    /* Possible comment start */
    if (c == '/') {
```

```

next = fgetc(src);

/* Single-line comment */
if (next == '/') {
    while ((c = fgetc(src)) != '\n' && c != EOF);
}

/* Multi-line comment */
else if (next == '*') {
    char prev = 0;
    while ((c = fgetc(src)) != EOF) {
        if (prev == '*' && c == '/')
            break;
        prev = c;
    }
}

/* Not a comment */
else {
    fputc(c, dst);
    fseek(src, -1, SEEK_CUR);
}

/* Normal character */
else {
    fputc(c, dst);
}

}

/* ===== PREPROCESSOR REMOVAL ===== */

void removePreprocessor(FILE *src, FILE *dst) { char c; while ((c = fgetc(src)) != EOF) { if (c ==
'#') { while (c != '\n' && c != EOF) c = fgetc(src); } else { fputc(c, dst); } } }

/* ===== TOKEN HELPERS ===== */

int isArithmetic(char c) { return (c=='+'||c=='-'||c=='*'||c=='/'||c=='%'); }

int isSpecialSymbol(char c) { char s[] = ";,(){}[]"; for (int i = 0; s[i]; i++) if (c == s[i]) return 1;
return 0; }

int isIdentifier(char *w) { if (!isalpha(w[0]) && w[0] != '_') return 0; for (int i = 1; w[i]; i++) if
(!isalnum(w[i]) && w[i] != '_') return 0; return 1; }

int isNumber(char *w) { int dot = 0; for (int i = 0; w[i]; i++) { if (w[i] == '.') dot++; else if
(!isdigit(w[i])) return 0; } return dot <= 1; }

/* ===== GET NEXT TOKEN ===== */

Token getNextToken(FILE *fp) { Token t; char c, buffer[50]; int i = 0; static int row = 1, col = 0;

```

```

t.lexeme[0] = '\0';

while ((c = fgetc(fp)) != EOF) {
    col++;

    if (c == '\n') {
        row++;
        col = 0;
        continue;
    }

    if (isspace(c)) continue;

    t.row = row;
    t.col = col;

    /* String literal (ignored content) */
    if (c == '"') {
        while ((c = fgetc(fp)) != '"' && c != EOF);
        strcpy(t.lexeme, "\"...\"");
        strcpy(t.type, "STRING_LITERAL");
        return t;
    }

    /* Identifier / Keyword */
    if (isalpha(c) || c == '_') {
        buffer[i++] = c;
        while (isalnum(c = fgetc(fp)) || c == '_') {
            buffer[i++] = c;
            col++;
        }
        buffer[i] = '\0';
        fseek(fp, -1, SEEK_CUR);

        strcpy(t.lexeme, buffer);
        strcpy(t.type, isKeyword(buffer) ? "KEYWORD" : "IDENTIFIER");
        return t;
    }

    /* Number */
    if (isdigit(c)) {
        buffer[i++] = c;
        while (isdigit(c = fgetc(fp)) || c == '.') {
            buffer[i++] = c;
            col++;
        }
        buffer[i] = '\0';
        fseek(fp, -1, SEEK_CUR);

        strcpy(t.lexeme, buffer);
        strcpy(t.type, "NUMBER");
        return t;
    }

```

```

}

/* Arithmetic operator */
if (isArithmetic(c)) {
    t.lexeme[0] = c;
    t.lexeme[1] = '\0';
    strcpy(t.type, "ARITHMETIC_OP");
    return t;
}

/* Relational operator */
if (c=='<'||c=='>'||c=='='||c=='!') {
    char n = fgetc(fp);
    buffer[0] = c;
    buffer[1] = n;
    buffer[2] = '\0';

    if (!strcmp(buffer,"<=")||!strcmp(buffer,">=")||
        !strcmp(buffer,"==")||!strcmp(buffer,"!=")) {
        strcpy(t.lexeme, buffer);
    } else {
        fseek(fp, -1, SEEK_CUR);
        t.lexeme[0] = c;
        t.lexeme[1] = '\0';
    }
    strcpy(t.type, "RELATIONAL_OP");
    return t;
}

/* Special symbol */
if (isSpecialSymbol(c)) {
    t.lexeme[0] = c;
    t.lexeme[1] = '\0';
    strcpy(t.type, "SPECIAL_SYMBOL");
    return t;
}
}

strcpy(t.type, "EOF");
return t;

}

```

Main Analyzer(analyser.c):

```
#include <stdio.h> #include <string.h> #include <stdlib.h>

#include "global_table.h" #include "local_table.h"

typedef struct { char lexeme[50]; char type[30]; int row; int col; } Token;

void removeComments(FILE *, FILE *); void removePreprocessor(FILE *, FILE *); Token getNextToken(FILE *);

char currentFunction[50] = ""; int insideFunction = 0; char lastType[20] = "";

int main() { FILE *src, *noComments, *clean; Token t;

initGlobalTable();
initLocalTables();

src = fopen("input.c", "r");
noComments = fopen("nocomments.c", "w");
clean = fopen("clean.c", "w");

if (!src || !noComments || !clean) {
    printf("Error opening files.\n");
    return 1;
}

removeComments(src, noComments);
fclose(src);
fclose(noComments);

noComments = fopen("nocomments.c", "r");
removePreprocessor(noComments, clean);
fclose(noComments);
fclose(clean);

clean = fopen("clean.c", "r");
if (!clean) {
    printf("Error opening cleaned file.\n");
    return 1;
}

while (1) {
    t = getNextToken(clean);
    if (strcmp(t.type, "EOF") == 0)
        break;

    if (strcmp(t.type, "KEYWORD") == 0 &&
        (!strcmp(t.lexeme, "int") ||
         !strcmp(t.lexeme, "bool") ||
         !strcmp(t.lexeme, "void"))) {
```

```

    strcpy(lastType, t.lexeme);
}

if (strcmp(t.type, "IDENTIFIER") == 0) {
    Token next = getNextToken(clean);

    if (!insideFunction && strcmp(next.lexeme, "(") == 0) {
        insertGlobal(t.lexeme, lastType, "function");
        createLocalTable(t.lexeme);
        strcpy(currentFunction, t.lexeme);
        insideFunction = 1;
    }
    else if (!insideFunction) {
        insertGlobal(t.lexeme, lastType, "variable");
    }
    else {
        insertLocal(currentFunction, t.lexeme, lastType);
    }

    fseek(clean, -strlen(next.lexeme), SEEK_CUR);
}

if (strcmp(t.lexeme, "{") == 0 && insideFunction) {
    insideFunction = 0;
    currentFunction[0] = '\0';
}
}

fclose(clean);

displayGlobalTable();
displayLocalTables();

return 0;

}

```

Global_table.c

```

#include <stdio.h> #include <stdlib.h> #include <string.h> #include "global_table.h"

GSymbol *gtable[GSIZE];

static int hash(char *s) { int h = 0; for (int i = 0; s[i]; i++) h = (h + s[i]) % GSIZE; return h; }

void initGlobalTable() { for (int i = 0; i < GSIZE; i++) gtable[i] = NULL; }

void insertGlobal(char *name, char *type, char *category) { int idx = hash(name); GSymbol *t = gtable[idx];

while (t) {
    if (strcmp(t->name, name) == 0)
        return;
    t = t->next;
}

```

```

GSymbol *n = malloc(sizeof(GSymbol));
strcpy(n->name, name);
strcpy(n->type, type);
strcpy(n->category, category);
n->next = gtable[idx];
gtable[idx] = n;

}

void displayGlobalTable() { printf("\nGLOBAL SYMBOL TABLE\n"); printf("-----\n"); printf("NAME\t\tTYPE\t\tCATEGORY\n");
printf("-----\n");

for (int i = 0; i < GSIZE; i++) {
    GSymbol *t = gtable[i];
    while (t) {
        printf("%-12s %-12s %s\n",
            t->name, t->type, t->category);
        t = t->next;
    }
}

}

```

Global_table.h

```

#ifndef GLOBAL_TABLE_H #define GLOBAL_TABLE_H

#define GSIZE 31

typedef struct GSymbol { char name[50]; char type[20]; char category[10]; // function / variable struct GSymbol *next; } GSymbol;

void initGlobalTable(); void insertGlobal(char *name, char *type, char *category); void displayGlobalTable();

#endif

```

local_table.h

```

#ifndef LOCAL_TABLE_H #define LOCAL_TABLE_H

#define LSIZE 31

typedef struct LSymbol { char name[50]; char type[20]; int size; struct LSymbol *next; } LSymbol;

typedef struct LocalTable { char func[50]; LSymbol *table[LSIZE]; struct LocalTable *next; } LocalTable;

void initLocalTables(); void createLocalTable(char *func); void insertLocal(char *func, char *name, char *type); void displayLocalTables();

#endif

```

local_table.c

```

#include <stdio.h> #include <stdlib.h> #include <string.h> #include "local_table.h"

static LocalTable *head = NULL;

static int hash(char *s) { int h = 0; for (int i = 0; s[i]; i++) h = (h + s[i]) % LSIZE; return h; }

static LocalTable* findTable(char *func) { LocalTable *t = head; while (t) { if (strcmp(t->func, func) == 0) return t; t = t->next; } return NULL; }

```

```

void initLocalTables() { head = NULL; }

void createLocalTable(char *func) { LocalTable *lt = malloc(sizeof(LocalTable)); strcpy(lt->func, func); for (int i = 0; i < LSIZE; i++) lt->table[i] = NULL; lt->next
= head; head = lt; }

void insertLocal(char *func, char *name, char *type) { LocalTable *lt = findTable(func); if (!lt) return;

int idx = hash(name);
LSymbol *t = lt->table[idx];

while (t) {
    if (strcmp(t->name, name) == 0)
        return;
    t = t->next;
}

LSymbol *n = malloc(sizeof(LSymbol));
strcpy(n->name, name);
strcpy(n->type, type);
n->size = 4;
n->next = lt->table[idx];
lt->table[idx] = n;

}

void displayLocalTables() { LocalTable *lt = head;

while (lt) {
    printf("\nLOCAL SYMBOL TABLE : %s\n", lt->func);
    printf("-----\n");
    printf("NAME\tTYPE\tSIZE\n");

    for (int i = 0; i < LSIZE; i++) {
        LSymbol *t = lt->table[i];
        while (t) {
            printf("%-8s %-6s %d\n",
                t->name, t->type, t->size);
            t = t->next;
        }
        lt = lt->next;
    }

}
}

```


INPUT:(input.c)

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int isKeyword(char *w) {
    char *k[] = {
        "auto", "break", "case", "char", "const", "continue", "default",
        "do", "double", "else", "enum", "extern", "float", "for", "goto",
        "if", "int", "long", "register", "return", "short", "signed",
        "sizeof", "static", "struct", "switch", "typedef", "union",
        "unsigned", "void", "volatile", "while"
    };

    int l = 0, r = 31, m, c;

    while (l <= r) {
        m = l + (r - l) / 2;
        c = strcmp(w, k[m]);
        if (c == 0) return 1;
        if (c < 0) r = m - 1;
        else l = m + 1;
    }
    return 0;
}

int main(int argc, char *argv[]) {
    FILE *f = fopen(argv[1], "r");
    char ch, w[100];
    int i = 0, j;

    while ((ch = fgetc(f)) != EOF) {
        if (isalnum(ch) || ch == '_') {
            w[i++] = ch;
        } else {
            if (i > 0) {
                w[i] = '\0';
                if (isKeyword(w)) {
                    for (j = 0; w[j]; j++)
                        w[j] = toupper(w[j]);
                }
                printf("%s", w);
                i = 0;
            }
            printf("%c", ch);
        }
    }
}
```

```
fclose(f);  
return 0;  
}
```

Output

➤ ./lexer

GLOBAL SYMBOL TABLE

NAME	TYPE	CATEGORY
isKeyword	int	function
c	int	variable
printf	int	function
i	int	variable
strcmp	int	function
l	int	variable
fclose	int	function
m	int	variable
main	int	function
r	int	variable
w	int	variable

LOCAL SYMBOL TABLE : fclose

NAME	TYPE	SIZE
f	int	4

LOCAL SYMBOL TABLE : printf

NAME	TYPE	SIZE
ch	int	4

LOCAL SYMBOL TABLE : printf

NAME	TYPE	SIZE
i	int	4
w	int	4

LOCAL SYMBOL TABLE : isKeyword

NAME	TYPE	SIZE
toupper	int	4
j	int	4
w	int	4

LOCAL SYMBOL TABLE : main

NAME	TYPE	SIZE
EOF	int	4
_	int	4
fopen	int	4
f	int	4
FILE	int	4
argc	int	4
i	int	4
j	int	4
isalnum	int	4
ch	int	4
fgetc	int	4
w	int	4
argv	int	4

LOCAL SYMBOL TABLE : strcmp

NAME	TYPE	SIZE
c	int	4
k	int	4

l int 4
m int 4
r int 4
w int 4

LOCAL SYMBOL TABLE : isKeyword

NAME	TYPE	SIZE
k	int	4
w	int	4